Project Report

Oak Hold'em

A multiplayer Texas Hold'em game for a local network, written in Java

(Poker Engine with AI Agents)

Course: **Java Programming**

Faculty: **Dr. Sharmila C**

Submitted by

Name	Registration number
Saksham Dubey	24BBS0081
Aryaman Sodha	24BBS0110
Tanay Doshi	24BBS0104

Source code: https://github.com/SakD2006/holdem

October 2026

Abstract

Oak Hold'em is a desktop poker game in which friends on the same local network (a home Wi-Fi or a college lab) play No-Limit Texas Hold'em against each other and against computer players. One computer runs the server and the database; every other player starts the desktop app, finds the server, signs in, and either creates a room, which gives a six-character room code, or joins one with a code. The room's creator starts the game.

The project is written in Java 21 as six Maven modules. A pure rules engine deals the cards, runs the betting, splits side pots and ranks hands. A Tomcat server holds the rooms, talks to the players over WebSockets, saves every hand to PostgreSQL through JDBC and to text files, and serves a few JSP pages (leaderboard, room results, hand replays) to any browser on the network. A JavaFX app is what people play on. Computer players come in three levels, the strongest of which remembers how each opponent plays. The server never trusts the client and never sends a player anyone else's cards.

The project uses every topic of the course syllabus: packages, abstract classes, inheritance, polymorphism, user-defined exceptions, file handling, collections, JavaFX, JDBC, servlets, JSP and multithreading. It contains about 19,500 lines of application code and more than 470 automated tests.

Contents

Abstract **1**

1 Introduction **3**
1.1 The game . . . 3
1.2 Problem and aim . . . 3
1.3 Objectives . . . 3

2 System overview **3**

3 Project modules **4**
3.1 poker-common: shared vocabulary . . . 5
3.2 poker-engine: the rules of the game . . . 5
3.3 poker-ai: computer players . . . 6
3.4 poker-server: the host machine . . . 6
3.5 poker-client-fx: the desktop app . . . 7
3.6 poker-bot-client: test players . . . 7

4 Java concepts used **8**
4.1 Database . . . 8

5 Sample code **9**
5.1 Abstract class and polymorphism: room commands . . . 9
5.2 Inheritance: a person or a bot in a seat . . . 9
5.3 User-defined exceptions . . . 10
5.4 Multithreading: one thread owns each room . . . 11
5.5 Collections: the deck . . . 12
5.6 JDBC: saving a new account . . . 12
5.7 Servlet: logging in . . . 13
5.8 File handling: hand-history files . . . 14
5.9 JSP: the leaderboard page . . . 14
5.10 Keeping a bot's thinking off the room's thread . . . 15

6 Source code **15**

7 Screenshots **16**
7.1 Desktop app . . . 16
7.2 Web pages . . . 18

8 Testing **19**

9 Conclusion and future work **20**

1 Introduction

1.1 The game

Texas Hold'em is the most widely played form of poker. Each player is dealt two private cards (*hole cards*). Five shared cards (the *board*) are then dealt face up in three stages: three cards (the *flop*), one (the *turn*) and one more (the *river*). There is a round of betting before the flop and after each stage. In each round a player may fold, check, call, bet or raise. In *No-Limit* Hold'em a player may bet any amount up to all their chips. A hand ends when everyone but one player has folded, or at the *showdown*, where the best five-card hand made from a player's two cards and the board wins the pot. Hands rank, from strongest: straight flush, four of a kind, full house, flush, straight, three of a kind, two pair, one pair, high card.

1.2 Problem and aim

Playing poker with friends usually needs either a physical deck and chips or an online site that needs an internet account. The aim of this project is a small, self-contained program that a group can run in one room on one network, with no internet and no installation beyond Java.

1.3 Objectives

- Play correct No-Limit Hold'em for 2 to 9 players, including all-ins and side pots.
- Let players create and join rooms with a short code, with the creator as host.
- Keep the game fair: the server checks every action and never reveals hidden cards.
- Save accounts and every finished hand, and show a leaderboard and hand replays.
- Offer computer players for when there are not enough people.
- Use, visibly, each Java topic in the course syllabus.

2 System overview

Figure 1 shows how the parts fit together. The *host machine* runs the server and PostgreSQL. Each player's computer runs the desktop app, which talks to the server in two ways: plain HTTP requests carrying JSON for signing in and managing rooms, and a WebSocket connection for the game itself, over which the server pushes every event the moment it happens.

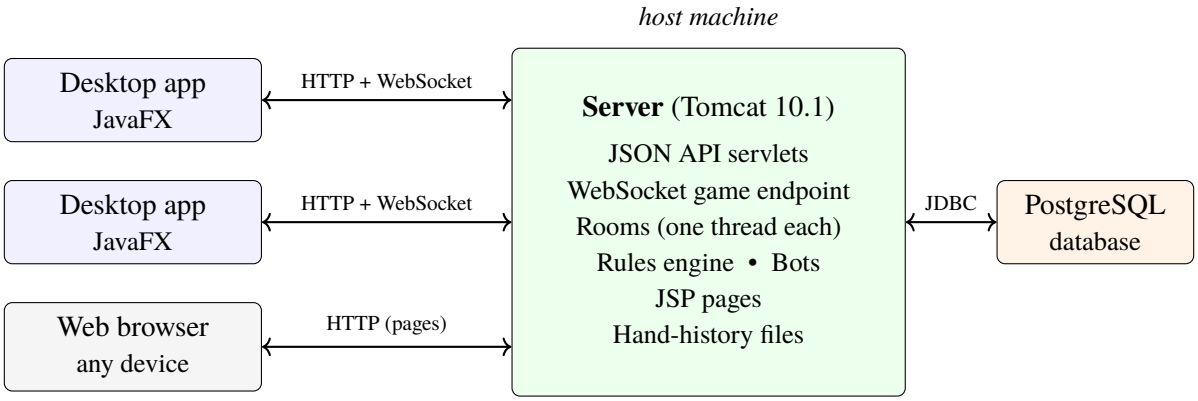


Figure 1: Architecture. Every player connects to the one server on the local network.

Four rules shape the design:

1. **The server decides.** Clients only ask ("I raise to 600"); the server checks the request against the rules before anything changes.
2. **Hidden cards stay hidden.** A player's hole cards are sent only to that player until the showdown. This filtering happens in one class, `EventRouter`.
3. **One thread per room.** Only a room's own thread changes that room, so the game logic needs no locks.
4. **Bots see only what a person would.** A computer player receives exactly the messages a person in its seat would receive.

Table 1: Technology used.

Area	Technology
Language and build	Java 21, Maven (multi-module, with the Maven wrapper)
Server	Apache Tomcat 10.1: Servlet 6, JSP 3.1 with JSTL 3, WebSocket 2.1
Database	PostgreSQL 16, JDBC with the HikariCP connection pool
Desktop app	JavaFX 21; the JDK's `java.net.http` client for HTTP and WebSocket
Data format	JSON through the Jackson library
Testing	JUnit 5 and AssertJ
Logging	SLF4J with Logback

3 Project modules

The code is split into six Maven modules. Each one is a separate folder with its own `pom.xml` and produces its own JAR (the server produces a WAR). The arrows in Figure 2 show which module uses which; nothing points the other way, so, for example, the rules engine knows nothing about networks, databases or windows.

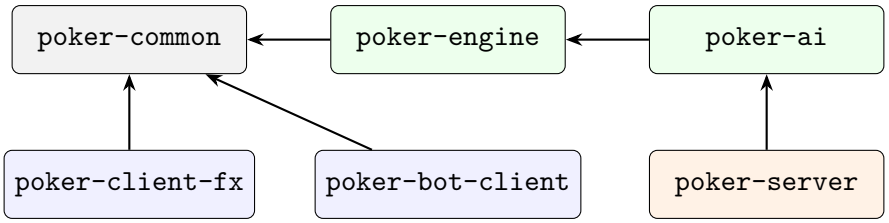


Figure 2: Module dependencies. An arrow means "uses".

Table 2: Size of each module (main code only; tests are separate).

Module	Java files	Lines	Test classes
`poker-common`	104	2,478	5
`poker-engine`	31	1,881	10
`poker-ai`	27	2,474	9
`poker-server`	74	6,182	26
`poker-client-fx`	43	5,725	12
`poker-bot-client`	5	759	1
Total	284	19,499	63

3.1 poker-common: shared vocabulary

What it is. The types that both the server and the clients need: cards, player actions, every message sent over the network, error codes and the exception classes. It has no input or output of its own.

Main packages. `card` (`Card`, `Rank`, `Suit`); `action` (the `PlayerAction` hierarchy: fold, check, call, bet, raise, all-in); `protocol` (the abstract `Message` class with about 40 client and server messages); `exception` (the `PokerException` tree); `lan` (the message format for finding a server on the network).

Usage. Because the server and the app use the very same message classes, Jackson turns a message into JSON on one side and back into the same Java object on the other.

3.2 poker-engine: the rules of the game

What it is. A pure implementation of No-Limit Hold'em. It deals, runs each betting round, works out who may do what, builds the main pot and side pots, ranks hands and pays the winners.

Main packages. `card` (`Deck`, `DeckFactory`); `hand` (`Hand`, the state machine for one hand, and `BettingRound`); `pot` (`PotCalculator`); `eval` (`HandEvaluator`, `HandValue`); `event` (the `GameEvent` classes the engine reports); `rules` (legal actions and minimum raises).

Usage. The server creates a `Hand`, feeds it each player's action and receives a list of events. The engine uses no threads, clock, files or randomness of its own: the shuffled deck is passed in. This makes every hand repeatable in tests, and the total number of chips is checked to be the same before and after every hand.

3.3 poker-ai: computer players

What it is. The strategies that choose a bot's action, kept apart from any networking so they can be tested and measured on their own.

Main classes. The abstract `BotStrategy` with three subclasses: `RuleBasedStrategy` (Easy: decides from its starting cards and hand strength), `MonteCarloStrategy` (Medium: estimates its chance of winning by dealing out the unseen cards hundreds of times) and `AdaptiveStrategy` (Hard: also keeps an `OpponentBook` of how often each opponent folds, raises and bluffs, and adjusts to it). `TableObserver` builds the bot's picture of the hand from the same messages a person would receive. The `arena` package plays bots against each other at about 100,000 hands a second to measure their strength.

Usage. The server wraps a strategy in an `AiController` and seats it like a player.

3.4 poker-server: the host machine

What it is. A web application that runs on Tomcat on the host machine. It is the only part that changes the game, and the only part that talks to the database.

Main packages.

- `web`: servlets for the JSON API (register, log in, create and list rooms, leaderboard) built on the abstract `BaseServlet`, and the page servlets built on `PageServlet` that hand data to the JSP pages.
- `ws`: the WebSocket endpoint `GameEndpoint`; each player's `Connection` has its own queue of outgoing messages.
- `room`: `RoomManager`, `Room`, the `RoomCommand` classes, `RoomActor` (the thread that owns a room) and `EventRouter` (the hole-card filter).
- `player`: the abstract `SeatController` with `RemoteHumanController` for people and `AiController` for bots.
- `db`: the abstract `BaseDao` and one DAO per table, the database migrations, and `HandRecordWriter`, a background thread that saves finished hands.
- `io`: `HandHistoryFileWriter` (one text file per room per day) and `ServerConfig` (reads `server.properties`).
- `auth`: password hashing and login tokens. `lan`: answers "is there a server here?" broadcasts so apps can find the server without typing its address.

Usage. Started with one Maven command on the host machine; it listens on port 8080.

3.5 poker-client-fx: the desktop app

What it is. The JavaFX program people play on.

Main packages. `app` (`HoldemApp`, `SceneRouter` for moving between screens, `RoomSession`); `net` (`ApiClient` for HTTP, `GameSocket` for the WebSocket, `ServerDiscovery`); `state` (`RoomState`, the app's copy of the table, updated from server messages); `view` (one class per screen: connect, log in, home, create and join room, waiting room, `TableView`); `util` (sounds, friendly error messages).

Usage. Network replies arrive on background threads and are passed to the JavaFX thread with `Platform.runLater`, so the window never freezes. The build also produces a folder with start scripts for Windows, macOS and Linux that can be copied to any PC with Java.

3.6 poker-bot-client: test players

What it is. A command-line program that starts many simple bots which connect over the network exactly as the desktop app does.

Usage. Used to test the server under load: for example, six bots playing 500 hands with no pauses. It checks that chips are conserved and that every hand reached the database.

4 Java concepts used

Table 3: Where each syllabus topic appears in the code.

Topic	Where it is used
Packages	Every module has its own package tree under `com.saksham.poker`, e.g. `server.room`, `engine.pot`, `client.view`.
Abstract classes	`PlayerAction`, `GameEvent`, `Message`, `RoomCommand`, `SeatController`, `PokerException`, `BaseDao`, `BaseServlet`, `PageServlet`, `BotStrategy`.
Inheritance	All their subclasses; `AdaptiveStrategy` extends `MonteCarloStrategy` extends `BotStrategy`; the exception tree.
Polymorphism	`command.execute(room)` runs whichever command was queued; `controller.onActionRequested(...)` reaches a person or a bot; Jackson reads the right message subclass from JSON.
User-defined exceptions	`PokerException` → `GameRuleException`, `RoomException`, `AuthException`, `ProtocolException` and about 20 specific exceptions, each mapped to an error code the client shows.
File handling	Hand-history text files (`HandHistoryFileWriter`), `server.properties`, the app's saved settings and login.
Collections	`ArrayDeque` for the deck and turn order, `List<Pot>`, `Set<Integer>` of seats eligible for a pot, `EnumMap` in the hand evaluator, `ConcurrentHashMap` of rooms, `LinkedBlockingQueue` of room commands, JavaFX `ObservableList`.
JavaFX	The whole desktop app: scenes, layouts, CSS styling, animations of cards being dealt and turned over.
JDBC	DAOs with `PreparedStatement`, transactions when a hand is saved, a connection pool, numbered migration scripts.
Servlets	The JSON API, a login filter, a character-set filter and a `ServletContextListener` that starts and stops the server's services.
JSP	Home, leaderboard, room results and hand replay pages, using JSTL and custom tag files.
Multithreading	One thread per room, shared timer threads, a sender thread per connection, the background database writer, the network-discovery thread, bot-thinking threads, and the JavaFX thread with `Platform.runLater` in the app.

4.1 Database

The server creates its tables itself on first start, from numbered SQL migration files.

Table 4: Database tables.

Table	Holds
`users`	Accounts: username, hashed password, and whether the account is a bot.
`auth_tokens`	Login tokens, so a player stays signed in.
`rooms`	Each room: code, name, settings, host, when it opened and closed.
`hands`	Each finished hand: room, number, board, pot.
`hand_players`	Each player in a hand: seat, cards (if shown), chips won or lost.
`hand_actions`	Every action in a hand, in order, for replays.

5 Sample code

The listings below are taken from the project, with imports left out.

5.1 Abstract class and polymorphism: room commands

Everything that changes a room, from a player's bet to a turn timer running out, is a `RoomCommand`. The room's thread calls `execute` without knowing which kind of command it holds.

Code 1: `RoomCommand` (poker-server, `server.room`).

```
public abstract class RoomCommand {
    /** Carries the command out. Runs on the room's thread. */
    public abstract void execute(Room room) throws PokerException;

    /** Called when execute refuses the command; tells the user who sent it. */
    public void failed(Room room, PokerException reason) {
        long user = userId();
        if (user >= 0) {
            room.sendError(user, reason);
        }
    }

    /** The user who asked for this, or -1 for a command from the server. */
    public long userId() {
        return -1;
    }
}
```

5.2 Inheritance: a person or a bot in a seat

A room talks only to a `SeatController`. `RemoteHumanController` sends the message over the player's WebSocket; `AiController` hands it to a bot strategy.

Code 2: `SeatController` (poker-server, `server.player`).

```java
public abstract class SeatController {
    protected final long userId;

    protected SeatController(long userId) {
        this.userId = userId;
    }

    /** It is this player's turn. */
    public abstract void onActionRequested(ActionRequest request);

    /** Something happened that this player is allowed to see. */
    public abstract void onEvent(ServerMessage event);

    public abstract boolean isConnected();

    /** True for a computer player. */
    public boolean isBot() {
        return false;
    }
}
```

5.3 User-defined exceptions

Every game or room error is a checked exception that knows the error code to send back.

Code 3: `PokerException` and one subclass (poker-common).

```java
public abstract class PokerException extends Exception {
    protected PokerException(String message) {
        super(message);
    }
    /** The code the client receives for this error. */
    public abstract ErrorCode code();
}

public class NotYourTurnException extends GameRuleException {
    public NotYourTurnException(String message) {
        super(message);
    }
    @Override
    public ErrorCode code() {
        return ErrorCode.NOT_YOUR_TURN;
    }
}
```

5.4 Multithreading: one thread owns each room

Any thread may submit a command; the room's own thread takes them from a `LinkedBlockingQueue` one at a time, so the room's data is never touched by two threads at once.

Code 4: Part of `RoomActor` (poker-server, `server.room`).

```
private final LinkedBlockingQueue<RoomCommand> queue = new LinkedBlockingQueue<>();

public RoomActor(String code, ScheduledExecutorService timers) {
    this.code = code;
    this.timers = timers;
    this.executor = Executors.newSingleThreadExecutor(runnable -> {
        Thread thread = new Thread(runnable, "room-" + code);
        thread.setDaemon(true);
        return thread;
    });
}

private void loop() {
    while (running) {
        try {
            room.run(queue.take());          // waits until a command arrives
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }
}

public boolean submit(RoomCommand command) {   // called from any thread
    if (!running) {
        return false;
    }
    queue.add(command);
    return true;
}
```

5.5 Collections: the deck

Code 5: Part of `Deck` (poker-engine, `engine.card`).

```java
public final class Deck {
    public static final int SIZE = 52;
    private final Deque<Card> cards;

    public Deck(List<Card> order) {
        if (order.size() != SIZE || new HashSet<>(order).size() != SIZE) {
            throw new IllegalArgumentException("Need all 52 cards once.");
        }
        this.cards = new ArrayDeque<>(order);
    }

    public static List<Card> standardOrder() {
        List<Card> cards = new ArrayList<>(SIZE);
        for (Suit suit : Suit.values()) {
            for (Rank rank : Rank.values()) {
                cards.add(new Card(rank, suit));
            }
        }
        return cards;
    }

    public Card deal() {
        if (cards.isEmpty()) {
            throw new IllegalStateException("The deck has no cards left.");
        }
        return cards.removeFirst();
    }
}
```

5.6 JDBC: saving a new account

All SQL goes through `PreparedStatement`, so a username can never change the meaning of the query (no SQL injection). Try-with-resources closes the connection, statement and result set.

Code 6: Part of `UserDao` (poker-server, `server.db`).

```
public User create(String username, String passwordHash)
        throws UsernameTakenException {
    String sql = "INSERT INTO users (username, password_hash) "
            + "VALUES (?, ?) RETURNING " + COLUMNS;
    try (Connection connection = dataSource.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, username);
        statement.setString(2, passwordHash);
        try (ResultSet row = statement.executeQuery()) {
            row.next();
            return mapRow(row);
        }
    } catch (SQLException e) {
        if (isDuplicate(e)) {
            throw new UsernameTakenException("That username is taken.");
        }
        throw new PersistenceException("Could not save the account", e);
    }
}
```

5.7 Servlet: logging in

Code 7: `LoginServlet` (poker-server, `server.web`).

```
@WebServlet("/api/auth/login")
public class LoginServlet extends BaseServlet {
    @Override
    protected void handlePost(HttpServletRequest request,
            HttpServletResponse response) throws PokerException, IOException {
        AuthRequest body = readJson(request, AuthRequest.class);
        Session session = app().sessions().login(body.username(), body.password());
        UserInfo user = new UserInfo(session.user().id(),
                session.user().username());
        writeJson(response, HttpServletResponse.SC_OK,
                new AuthResponse(session.token(), user));
    }
}
```

5.8 File handling: hand-history files

Code 8: Part of `HandHistoryFileWriter` (poker-server, `server.io`).

```
public Path append(HandRecord hand) {
    // e.g. data/hand-history/room-ABC234/2026-10-22.txt
    Path file = fileFor(hand);
    try {
        Files.createDirectories(file.getParent());
        try (BufferedWriter writer = Files.newBufferedWriter(file, UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
            writer.write(format(hand));
        }
        return file;
    } catch (IOException e) {
        throw new StorageException("Could not write " + file, e);
    }
}
```

5.9 JSP: the leaderboard page

Code 9: Part of `leaderboard.jsp` (poker-server, `WEB-INF/views`).

```
<c:forEach var="entry" items="${entries}">
  <tr class="${entry.rank() == 1 ? 'first' : ''}">
    <td class="num"><c:out value="${entry.rank()}"/></td>
    <td><c:out value="${entry.username()}"/></td>
    <td class="num"><t:net value="${entry.totalNet()}"/></td>
    <td class="num">
      <fmt:formatNumber value="${entry.handsPlayed()}" groupingUsed="true"/>
    </td>
  </tr>
</c:forEach>
```

5.10 Keeping a bot's thinking off the room's thread

A room's thread must never wait. When it is a bot's turn, `AiController` takes a snapshot of what the bot can see and schedules the decision on a separate pool of "thinker" threads, after a short human-looking pause.

Code 10: Part of `AiController` (poker-server, `server.player`).

```
@Override
public void onActionRequested(ActionRequest request) {
    LegalActions legal = request.legal();
    // Taken now, on the room's thread, and never changed:
    // safe to hand to another thread.
    Observation seen = observer.observe(legal.canCheck(), legal.callAmount(),
            legal.canBet(), legal.canRaise(),
            legal.minRaiseTo(), legal.maxRaiseTo());
    long pause = thinkMinMs + random.nextLong(thinkMaxMs - thinkMinMs + 1);
    thinkers.schedule(() -> act(request.turnId(), seen),
            pause, TimeUnit.MILLISECONDS);
}
```

6 Source code

The complete source code, with build and run instructions, is on GitHub:

https://github.com/SakD2006/holdem

To run it on the host machine (with Java 21 and Docker installed):

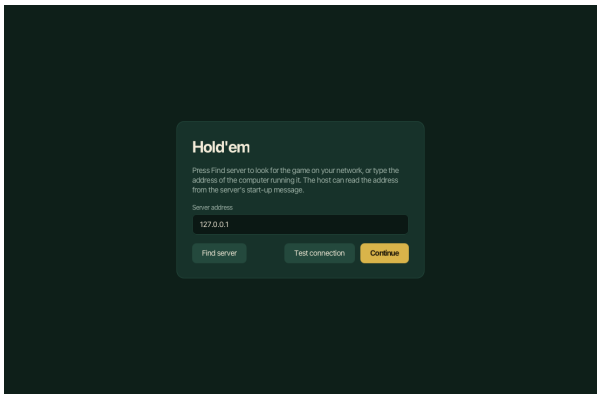
Code 11: Starting the database, the server and the app.

```
docker compose up -d                                          # PostgreSQL
./mvnw -pl poker-server -am -DskipTests package cargo:run     # server on port 8080
./mvnw -pl poker-client-fx -am compile javafx:run             # desktop app
```

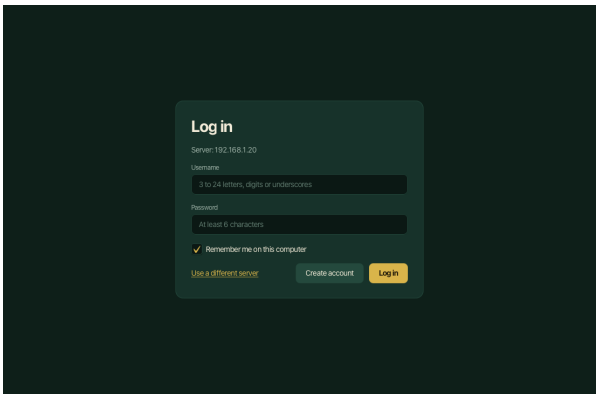
Other players only need the app folder and Java; they click *Find server* or type the host machine's IP address.

7 Screenshots

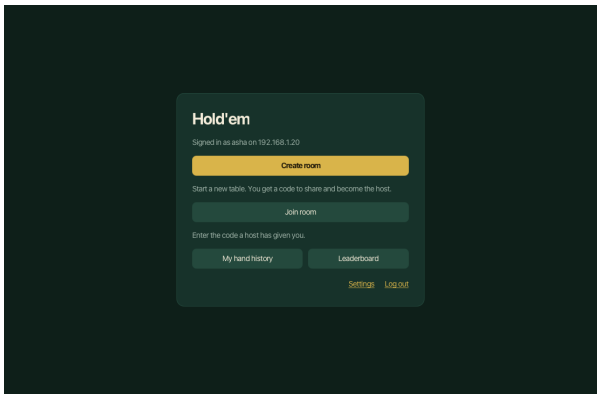
7.1 Desktop app



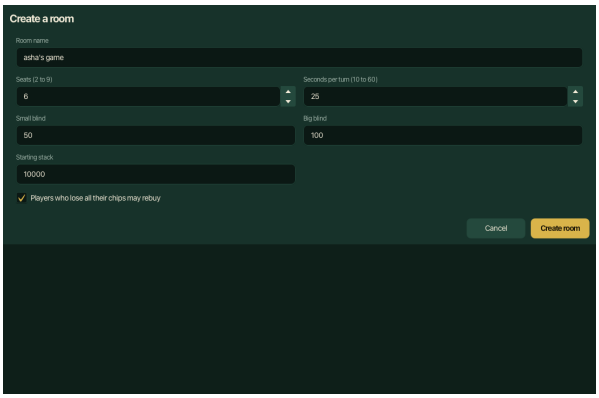
(a) Connecting to the server



(b) Logging in



(c) Home: create or join a room



(d) Room settings

Figure 3: Getting into a game.

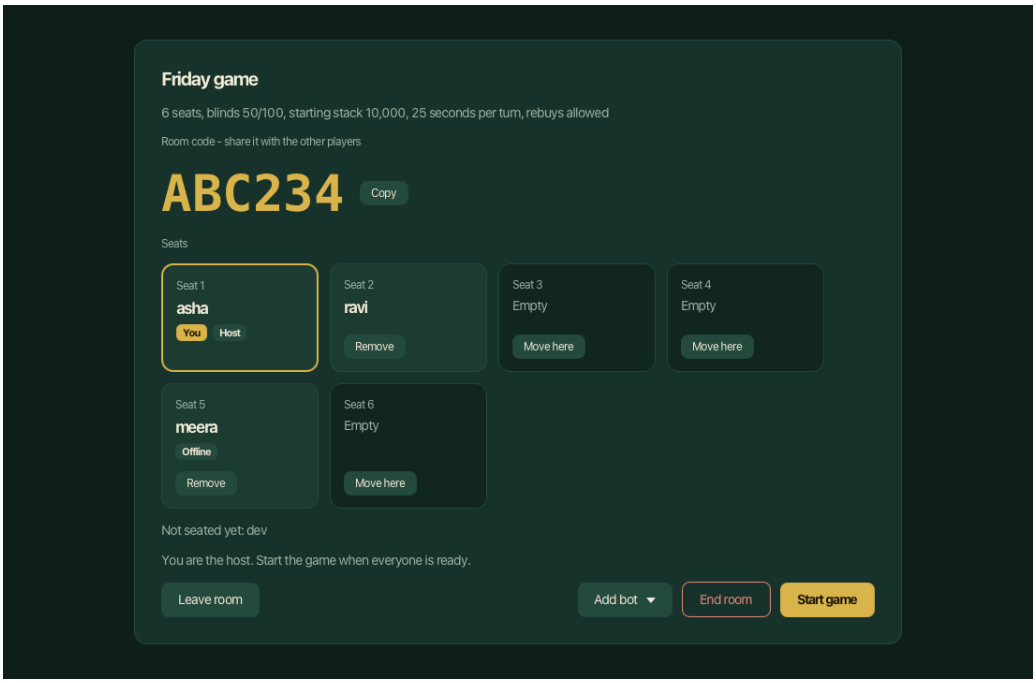


Figure 4: The waiting room seen by the host, with the room code to share, seat controls and the *Add bot* menu.



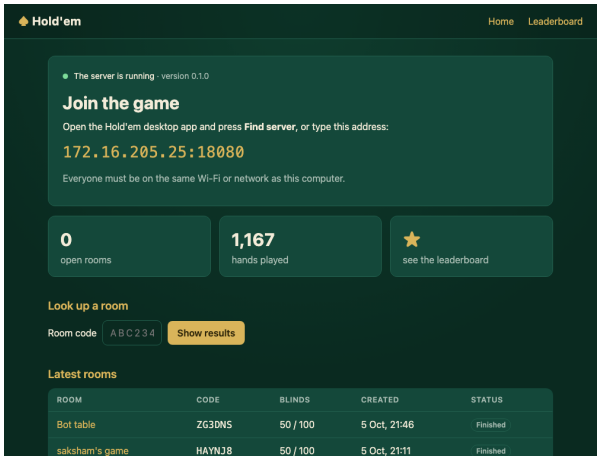
Figure 5: At the table on the player's turn: the action buttons and bet slider at the bottom, the hand log and chat on the right.



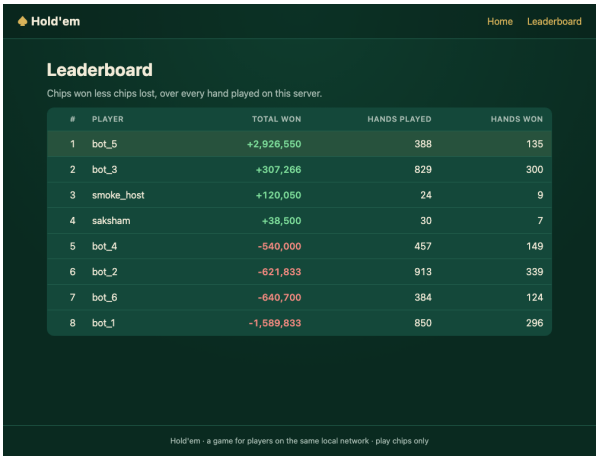
Figure 6: A showdown: the remaining players' cards are turned over and the winner is marked.

7.2 Web pages

Anyone on the network can open these in a browser at `http://<host-ip>:8080/poker/`.



(a) Home page



(b) Leaderboard

Figure 7: JSP pages served by the server.

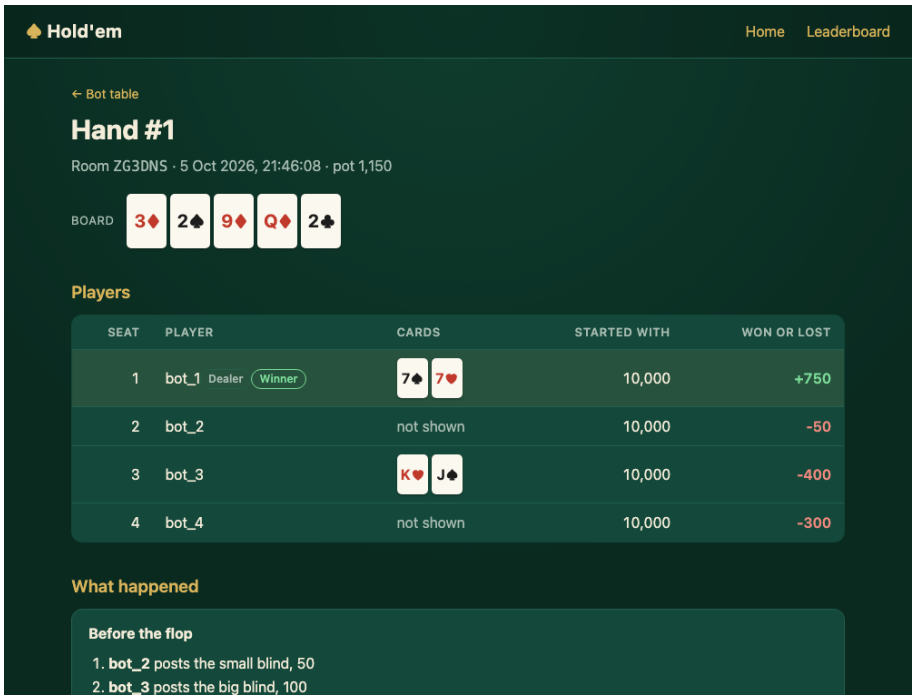


Figure 8: Replay of a saved hand, action by action.

8 Testing

The project has more than 470 automated JUnit tests, run with `./mvnw clean verify`.

- **Engine:** betting rules, minimum raises, all-ins, side pots with up to nine players, and the hand evaluator checked against every hand category. Each test hand uses a fixed deck, and chips are checked to be conserved.
- **Server:** rooms are driven command by command through a test harness, including reconnecting, kicking players, rebuys and the turn timer. A separate test checks that no message ever carries another player's hole cards.
- **Database:** DAO tests run against a real PostgreSQL in a temporary schema.
- **Bots:** a test seats real bots on real threads for hundreds of hands.
- **Load:** the bot client plays hundreds of hands over the network; afterwards the number of hands in the database must match.

9 Conclusion and future work

Oak Hold'em is a complete, working poker game for a local network. It plays correct No-Limit Texas Hold'em for up to nine players, keeps every player's cards private, saves every hand, and offers computer players at three levels. Splitting it into modules kept the game rules separate from the network, the database and the windows, which made each part easy to test on its own. Every topic of the Java syllabus appears in a place where it does real work.

Possible next steps are a tournament mode with rising blinds, spectators who can watch a table, and a stronger computer player built on game-theory methods.